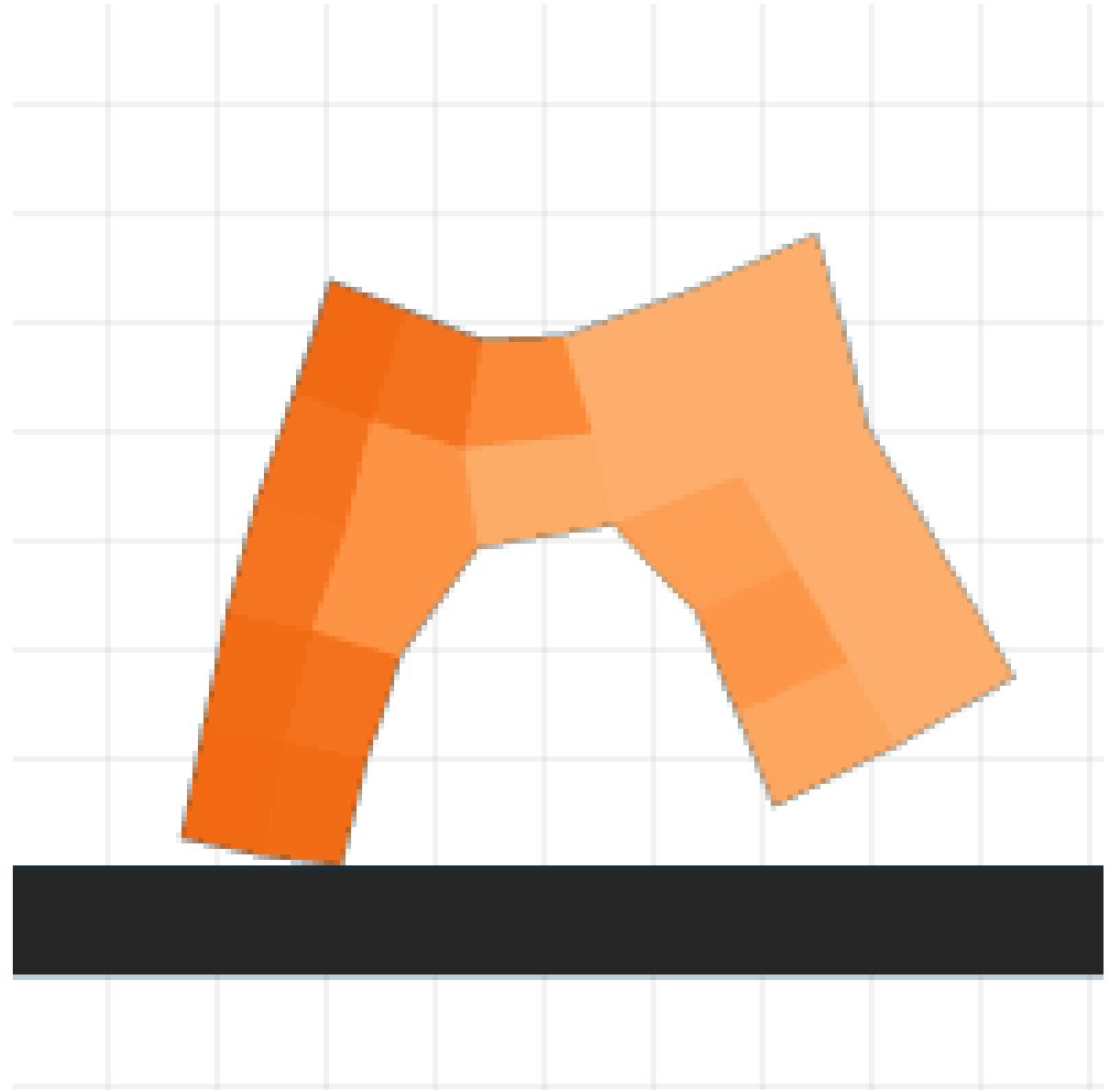


Evolution of Agents

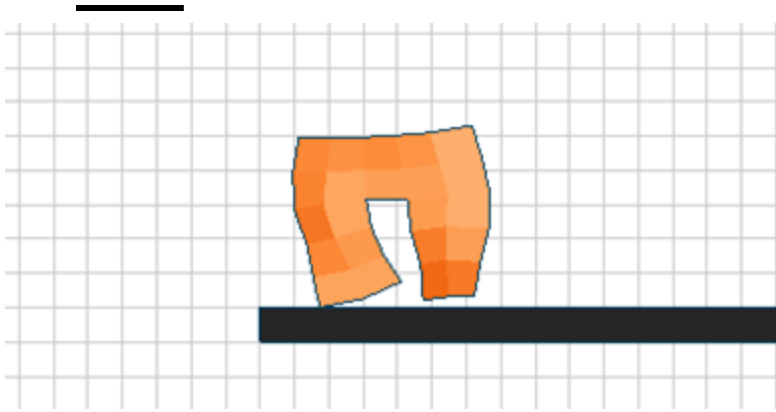
AL AZZI Abdo

ARPHANT Gwenc'hlan



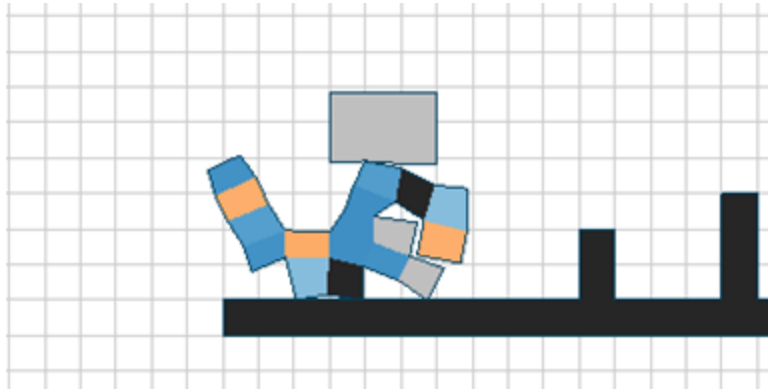
Environments

Walker



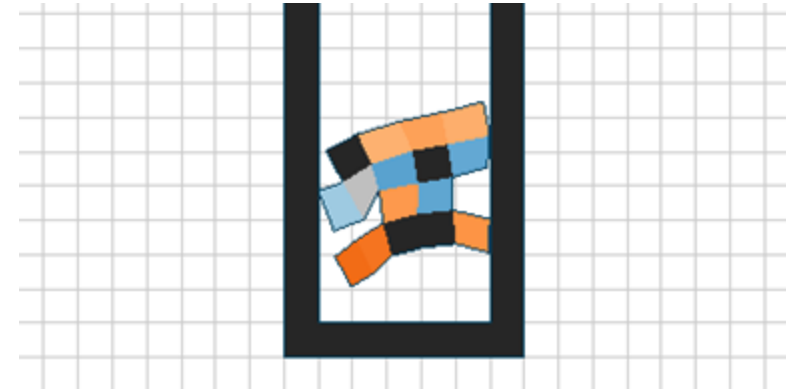
- Finding stability on flat surface is much easier, simpler dynamics, less inputs

Thrower



- Complex dynamics, interaction with object, require timing, balance, targeting as well as strength

Climber

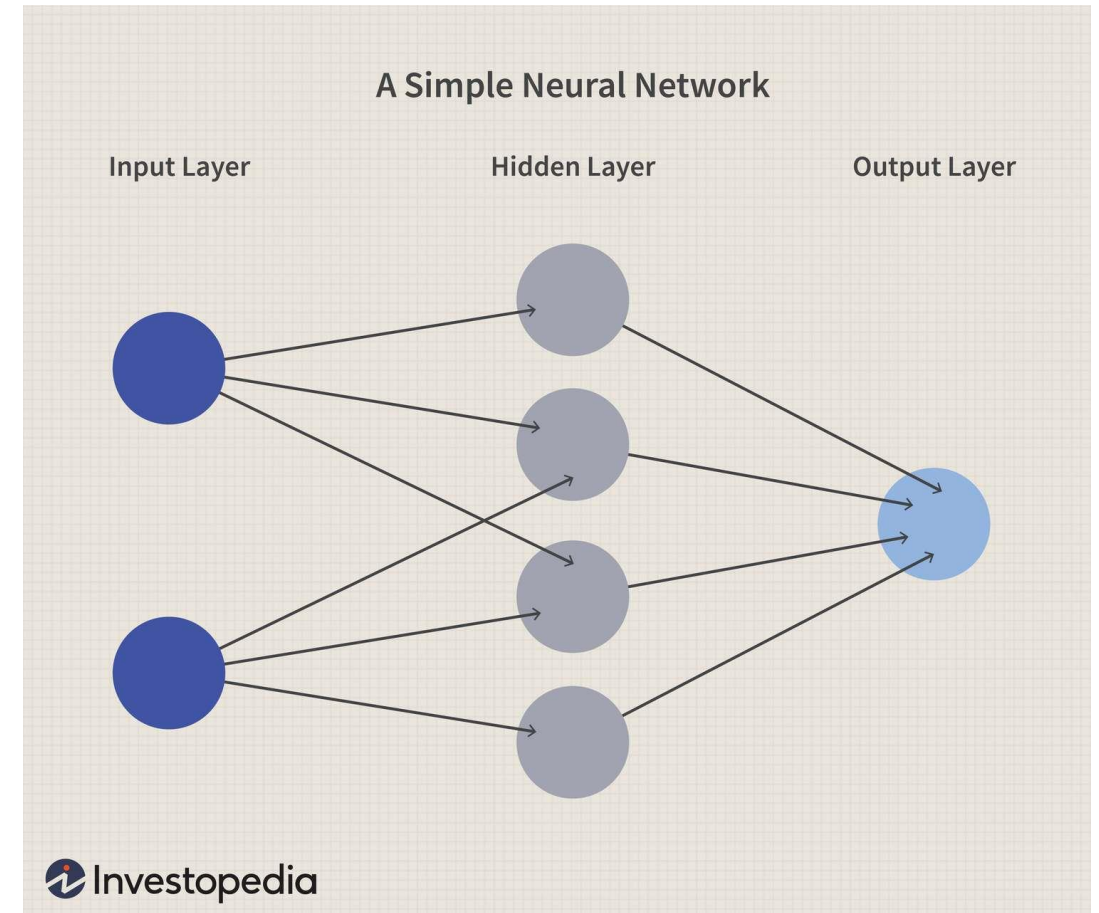


- Gravity fight, Difficult to find a balanced state as well as generating upwards thrust, again requires lots of timing and accuracy

- Inputs mainly affected by the degrees of freedom: ie voxel types of the model and size (affecting complexity of training due to interaction)
- Outputs determined by the task at hand

Network Configuration Choices

- Network is dependent on the geometry of the robot (number of inputs and outputs)
 - ◆ Different for each task
- Kept a Feedforward Neural Network consisting of n input layer, two fully connected hidden layers (of size h_size), and an output layer and ReLU activation functions.
- Improvements:
 - ◆ Try the simple 1 hidden layer structure for all final results (Reduction of computational costs & run times)



WHY ReLU ?

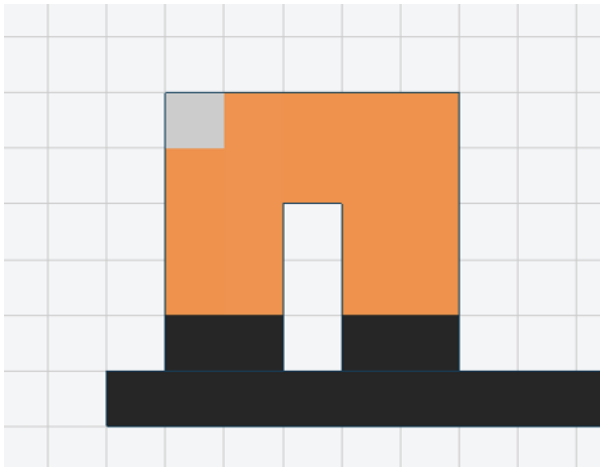
ReLU effectively handles non-linearities in the environment while avoiding the "vanishing gradient" problem commonly associated with deeper networks or tanh/sigmoid activations. It provides a strong, non-saturating signal for the agents to learn complex locomotion.

WHY Layer Normalization before ReLU?

Since evolutionary algorithms rely heavily on random mutations, Layer Normalization ensures that the outputs of hidden layers remain scaled and stable, preventing a single massive mutation from completely breaking the network's behavior.

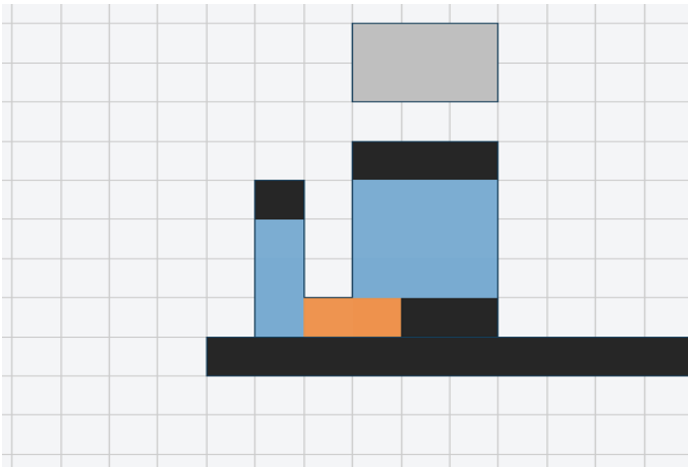
Model Designs

Walker



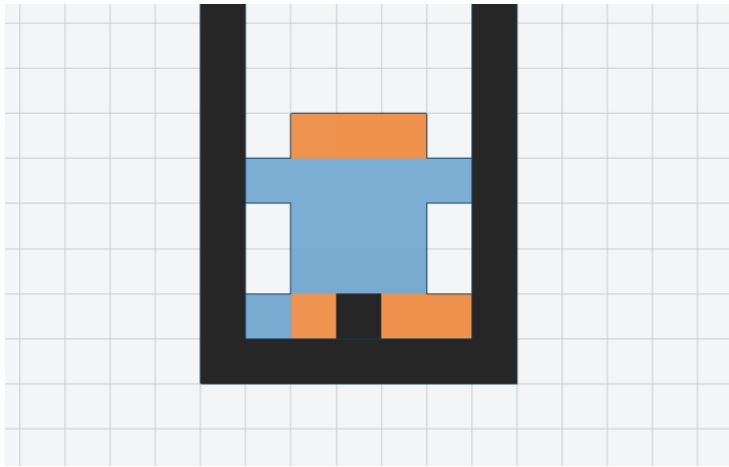
[3, 3, 3, 3, 3],
[3, 3, 3, 3, 3],
[3, 3, 0, 3, 3],
[3, 3, 0, 3, 3],
[1, 1, 0, 1, 1]

Thrower



[0, 0, 1, 1, 1],
[1, 0, 4, 4, 4],
[4, 0, 4, 4, 4],
[4, 0, 4, 4, 4],
[3, 3, 3, 1, 1]

Climber



[0, 3, 3, 3, 0]
[4, 4, 4, 4, 4]
[0, 4, 4, 4, 0]
[0, 4, 4, 4, 0]
[3, 3, 1, 3, 3]

The EV algorithm

- Multi-threading

```
num_workers = mp.cpu_count() - 1  
pool = mp.Pool(processes=num_workers)
```

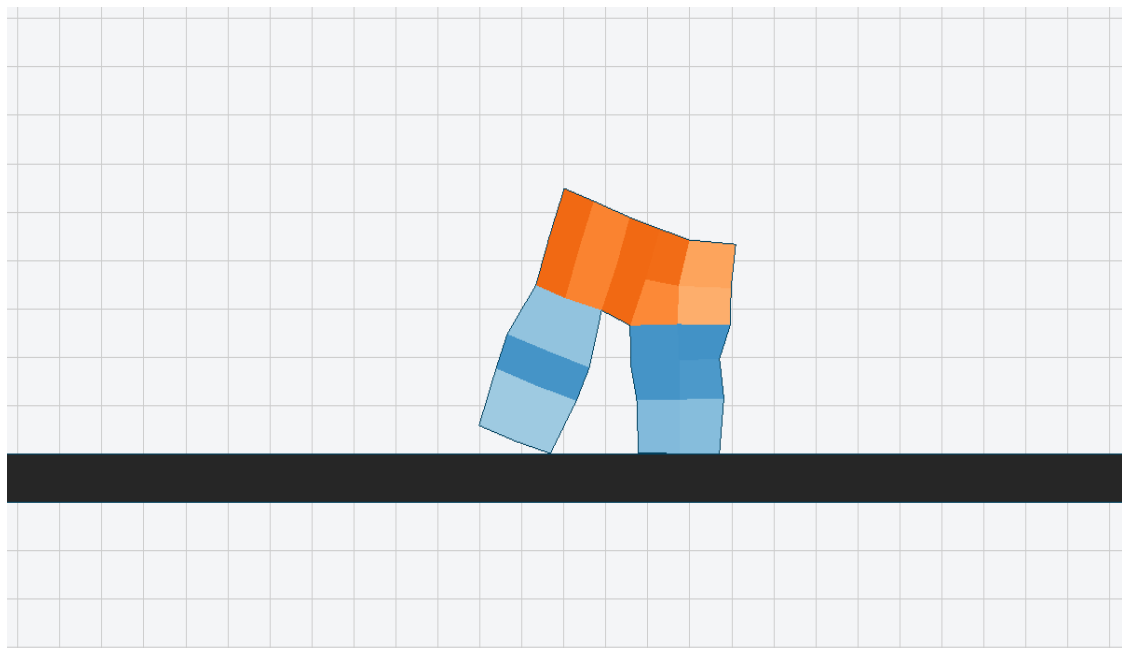
- Reducing mutation throw generation

```
sigma /= np.exp(sigma_lr)  
sigma = np.clip(sigma, 0.01, 1.0)
```

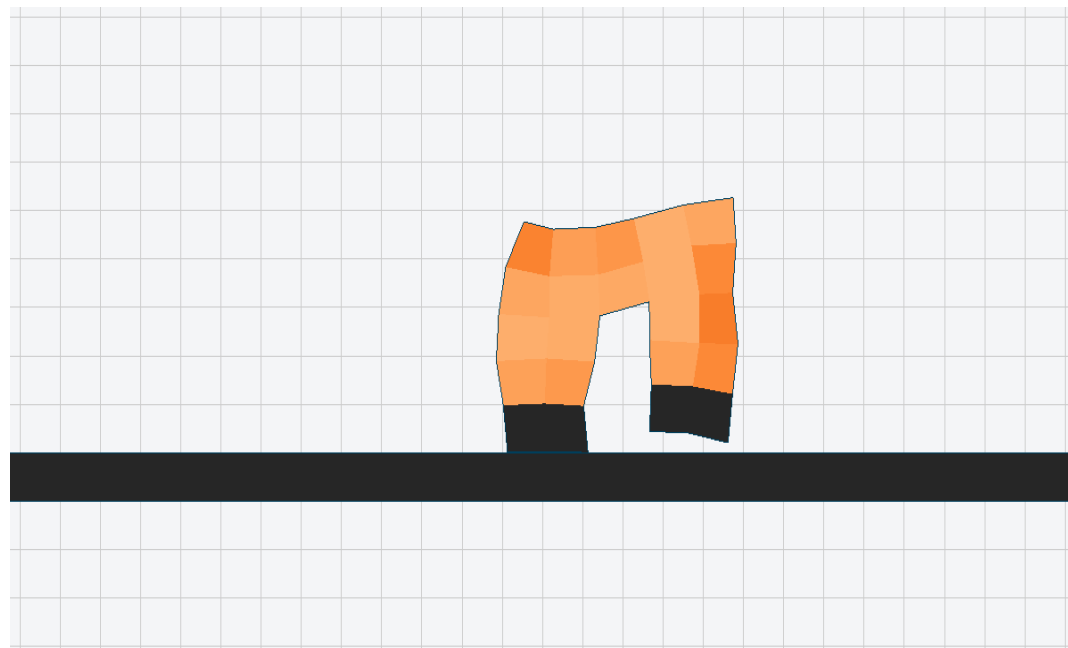
Results

Walker

params = [gens, lambda, mu, sigma, lr, steps]



First naive attempt



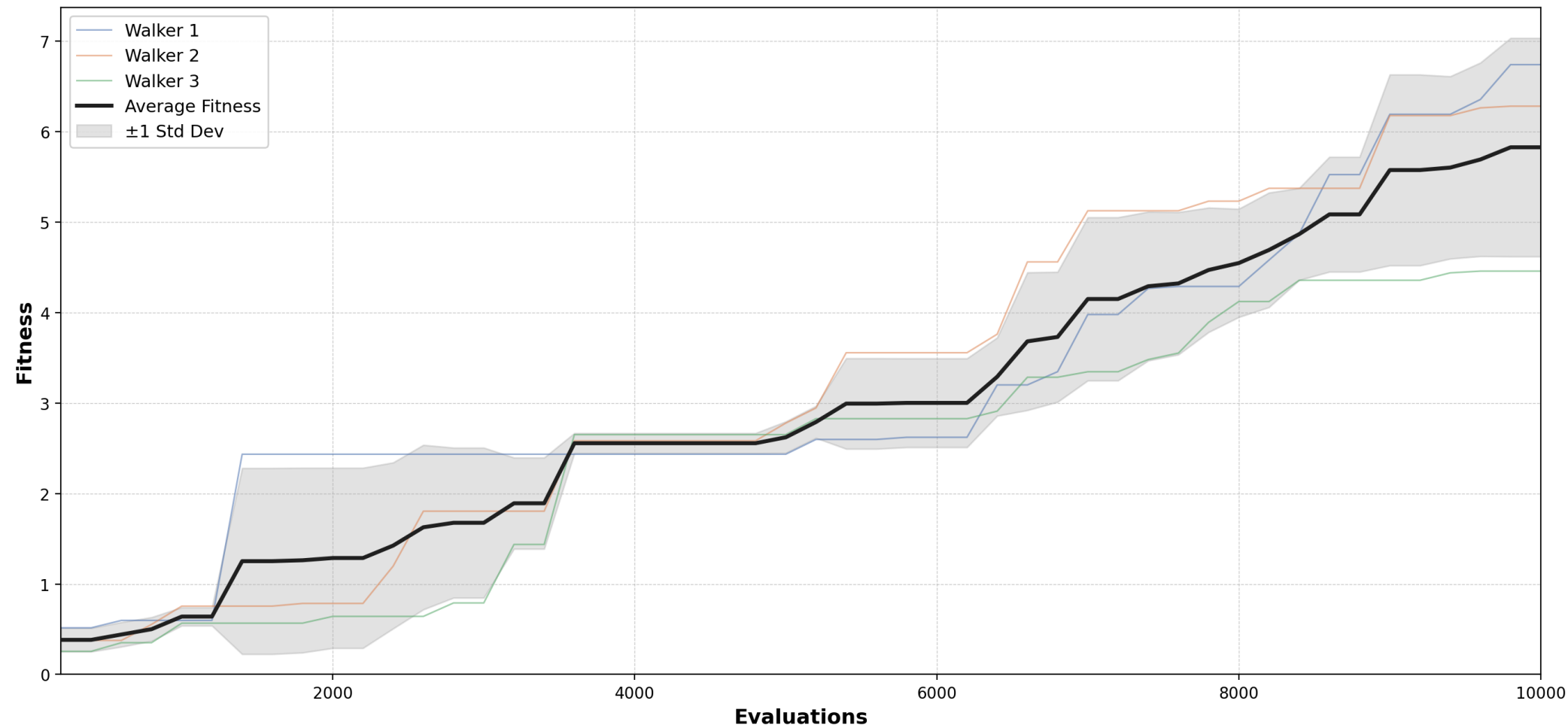
Second attempt

params = [50, 200, 100, 0.4, 0.1, 500]

Results

Walker

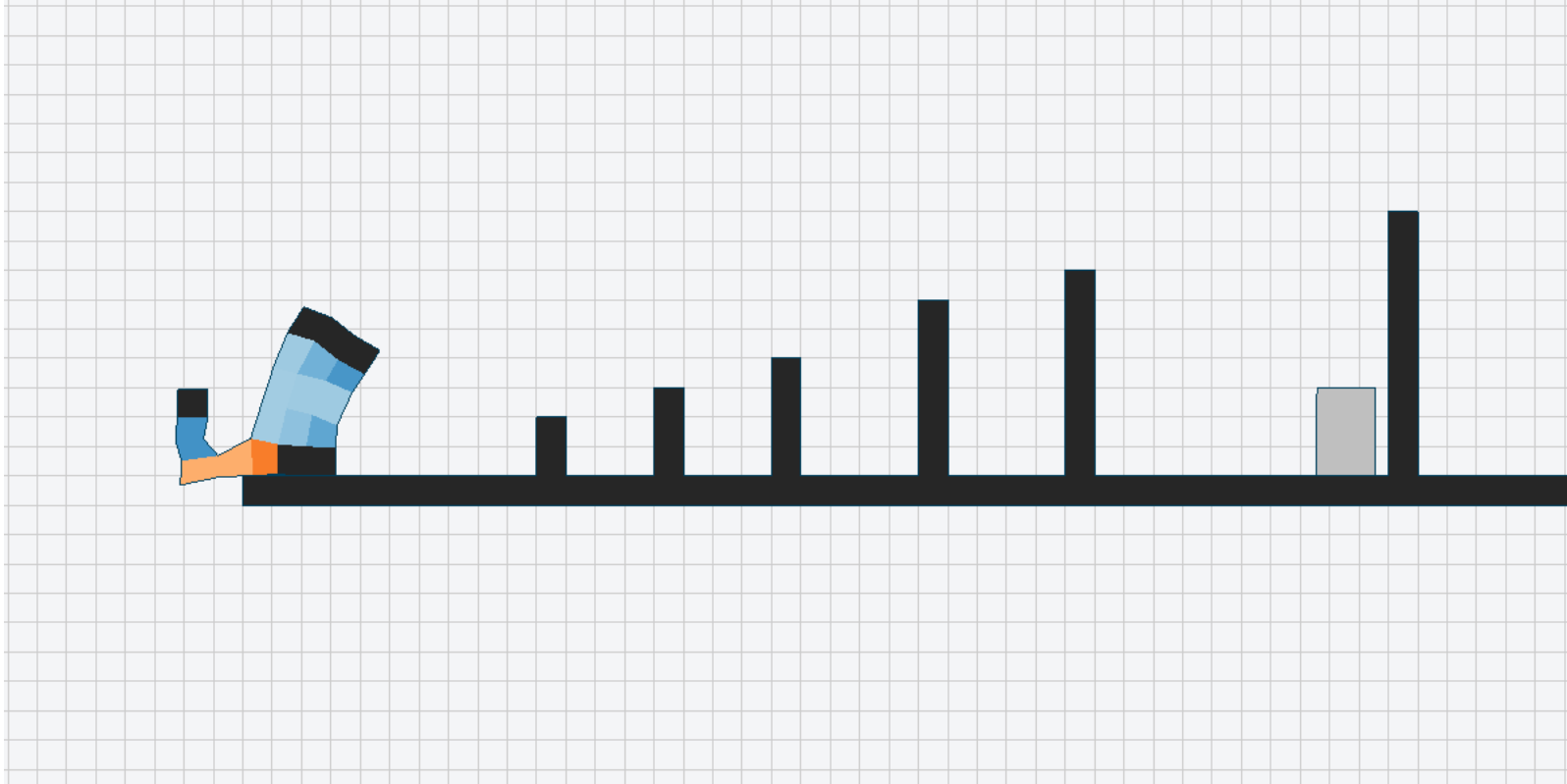
Average Walker Fitness with Variance Envelope



Results

Thrower

—

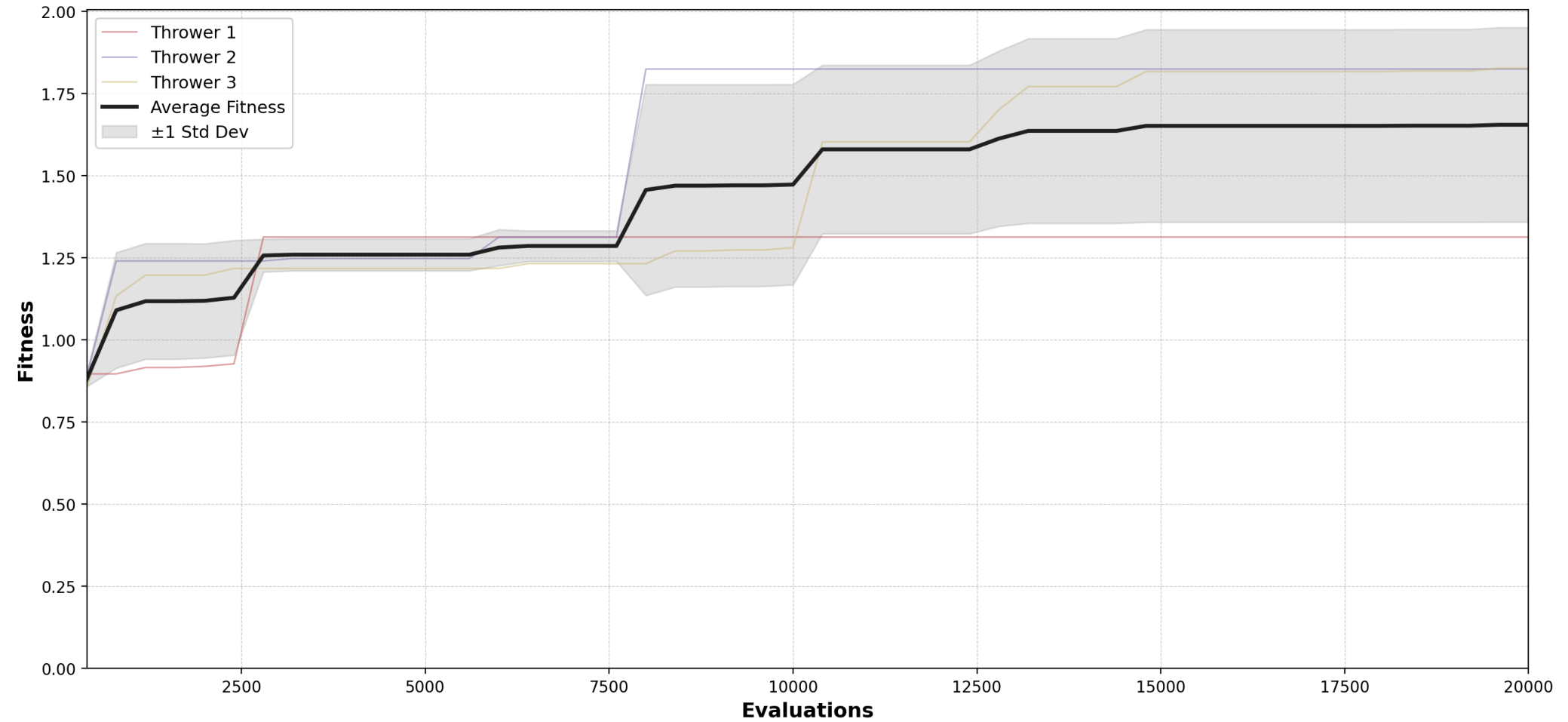


Results

Thrower

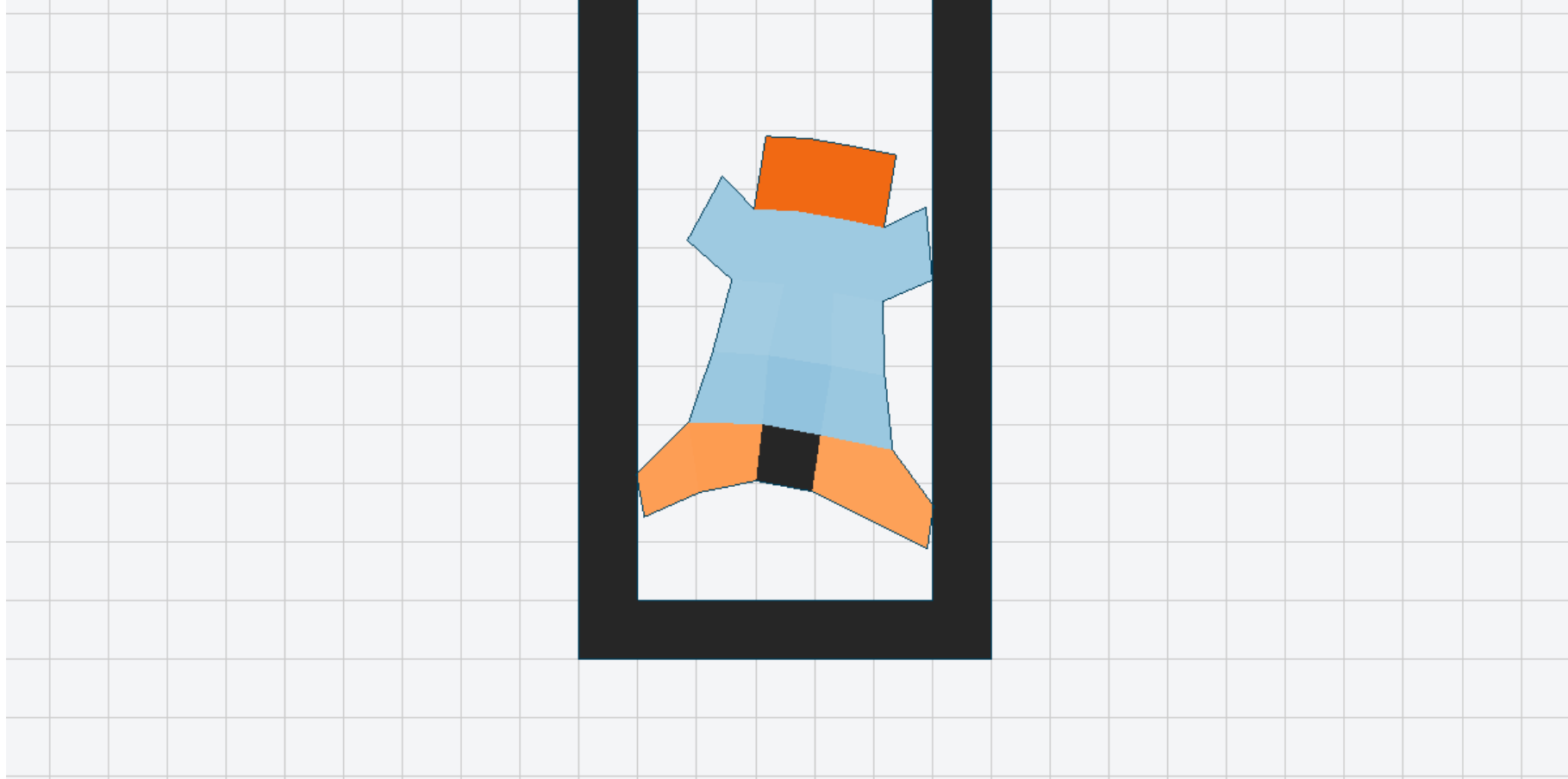
—

Average Thrower Fitness with Variance Envelope



Results

Climber

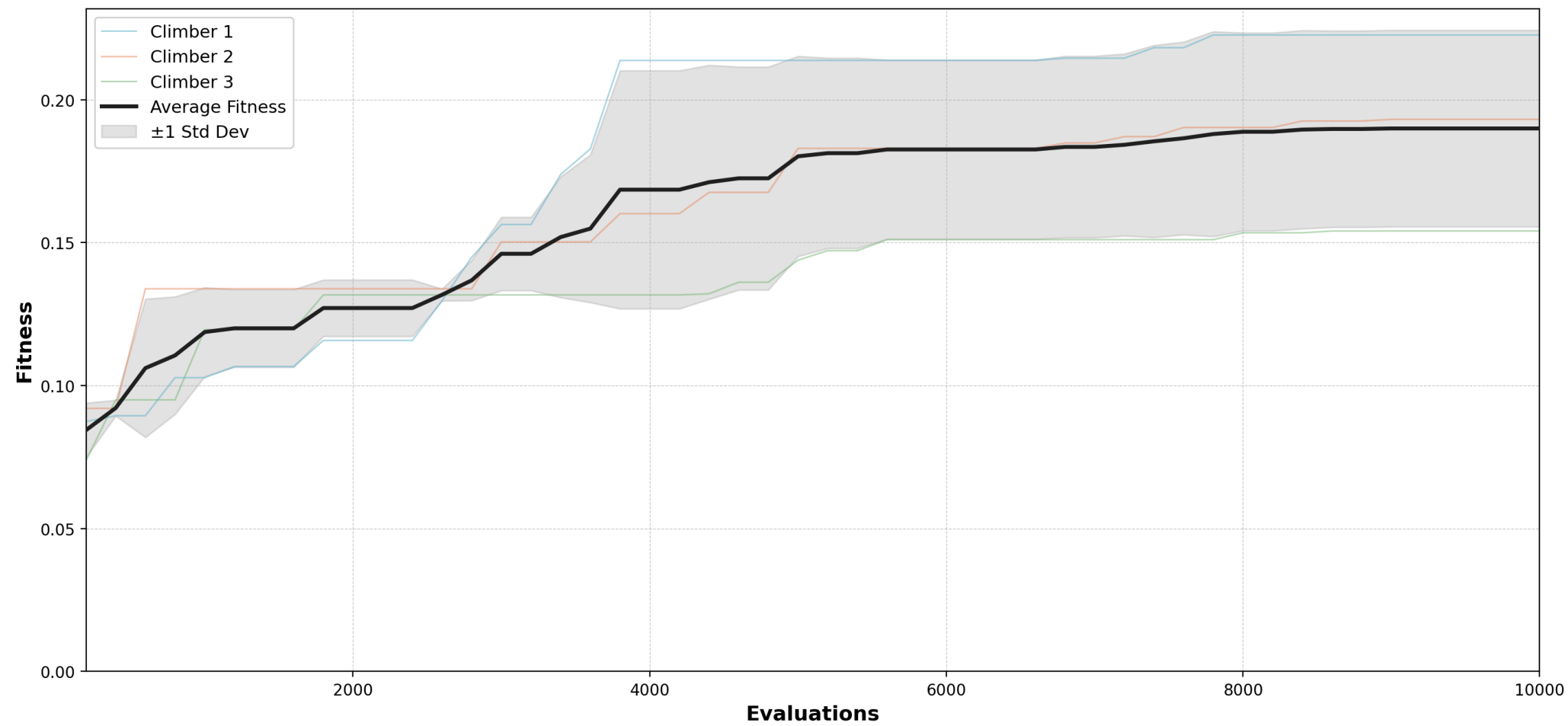


params = [50,200,100,0.4,0.1,500]

Results

Climber

Average Climber Fitness with Variance Envelope



Conclusion

How to
improve 😊